

- Darkjumper (<http://ignum.dl.sourceforge.net/project/darkjumper/darkjumper.v5.7.tar.gz>) will try to find every website that is hosted at the same server as your target, and will check vulnerabilities such as SQL injection, RFI, LFI, blind SQL injection, etc.
- Simple Local File Inclusion Exploiter (http://packetstorm.linuxsecurity.com/UNIX/scanners/lfi_spoiler.py.txt) is again a Python script that enables you to identify if a page is vulnerable to Local File Inclusion (LFI) attacks.

Checking for file inclusion vulnerabilities

To test whether your application has an RFI vulnerability, you can perform the following steps:

- Submit, in each targeted parameter, a URL for a resource on a Web server that you control, and determine whether any requests are received from the server hosting the target application.
- If the above test fails, try submitting a URL containing a non-existent IP address, and determine whether a time-out occurs while the server attempts to connect.

To test for local file inclusions, perform the following steps:

- Submit the name of a known executable resource in the input fields, and determine whether there is any change in the application's behaviour.
- Submit the name of a known static resource in the input fields, and determine whether its contents are copied into the application's response.
- If the application is vulnerable to LFI, attempt to access any sensitive functionality or resources that you cannot reach directly via the Web server.

File upload vulnerabilities

Any time you allow users to upload files—such as the image allowed by the guest-book application—you introduce vulnerability into your application. Users can upload a virus, root-kit, or other malicious script just as easily as an innocuous image file. File uploads can be a major security threat, and can be easily exploited to gain access to the file system of the server hosting the Web application. To understand more about file-upload attacks, let's start with a simple attack scenario.

Attack Scenario 3

Consider a Web application with the following PHP file upload script (*upload.php*):

```
<?php
$target_path = "uploads/";
$target_path = $target_path . basename( $_FILES['uploadedfile']
['name']);
if(move_uploaded_file($_FILES['uploadedfile']['tmp_name'],
$target_path))
```

```
{
echo "The file ". basename( $_FILES['uploadedfile']['name']).
" has been uploaded";
}
else
{
echo "There was an error uploading the file, please try again!";
}
?>
```

The above PHP script is handled by the HTML form below:

```
<form enctype="multipart/form-data" action="uploader.php"
method="POST">
<input type="hidden" name="MAX_FILE_SIZE" value="100000" />
Choose a file to upload: <input name="uploadedfile" type="file" /><br
/>
<input type="submit" value="Upload File" />
</form>
```

When PHP receives a POST request with the encoding type *multipart/form-data*, it will create a temporary file with a random name in a temp directory (e.g., */var/tmp/php4v1kbt*). PHP will also populate the global array `$_FILES` with information about the uploaded file:

`$_FILES['uploadedfile']['name']`: The original name of the file on the client machine.
`$_FILES['uploadedfile']['type']`: The MIME type of the file.
`$_FILES['uploadedfile']['size']`: The size of the file in bytes.
`$_FILES['uploadedfile']['tmp_name']`: The temporary filename in which the uploaded file was stored on the server.

The PHP function `move_uploaded_file` will move the temporary file to a location provided by the user. In this case, the destination is below the server root. Therefore, the files can be accessed using a URL like <http://www.example.com/uploads/uploadedfile.jpg>. In this simple example, there are no restrictions about the type of files allowed for upload, and therefore an attacker can upload a PHP or .NET file with malicious code that can lead to a server compromise. In such a case, an attacker can easily upload a malicious payload or a c99r/57 shell and get control over the server.

However, even if security measures are deployed to check and verify the file extension before uploading, the attacker can also bypass that using a null character `'%00'`. All one needs to do is select the PHP file to upload, without clicking the *Upload* button. The path of the selected file to upload will be showed, which for a *NIX system might be somewhat like */Users/username/exploit.php*, and for Windows systems, like *c:\exploit.php*. At this point, one should manually type the null character, followed by one of the image extensions, like */Users/username/exploit.php%00.jpg*, or *c:\exploit.php%00.jpg*.